

UNIVERSIDAD AUTÓNOMA DE QUERÉTARO

OPTIMIZACIÓN COMBINATORIA

Instructor: Dr. Arturo González Gutiérrez

Alumno: Ing. Daniel Eduardo González Alvarado

TAREA #2: Subsecuencia de Suma Máxima

1. Introducción

El problema de encontrar la subsecuencia de suma máxima consiste en determinar, dentro de un arreglo de números enteros, la subsecuencia contigua cuya suma de elementos sea la mayor posible. Este problema es clásico en el análisis de algoritmos, ya que permite comparar distintas estrategias de solución y analizar su complejidad computacional.

Para esto se implementaron en C++ cuatro algoritmos discutidos en clase para resolver el problema de la subsecuencia de suma máxima. Cada uno de estos algoritmos presenta una complejidad temporal distinta, lo que permite observar cómo el diseño algorítmico influye directamente en el tiempo de ejecución al aumentar el tamaño de la instancia.

Una vez verificado el funcionamiento correcto de las implementaciones, se realizaron experimentos para medir los tiempos de ejecución utilizando arreglos de diferentes tamaños. Las pruebas se llevaron a cabo para valores crecientes de n , comenzando desde $n = 10$ y aumentando progresivamente hasta tamaños mayores, manteniendo siempre la misma instancia de datos para todos los algoritmos con el fin de garantizar una comparación justa de los tiempos de ejecución.

Además de los cuatro algoritmos presentados en clase, se propuso un algoritmo adicional cuya eficiencia es al menos equivalente al mejor de los algoritmos analizados.

1.1. Características Computacionales

El equipo propio donde se corrieron los experimentos cuenta con las siguientes características computacionales:



Nombre del equipo: LAPTOP-EHFPNJP6
Sistema operativo: Windows 11 Home Single Language 64 bits (10.0, compilación 26200)
Idioma: español (configuración regional: español)
Fabricante del sistema: ASUSTeK COMPUTER INC.
Modelo del sistema: ASUS TUF Gaming F15 FX506HCB_FX506HCB
BIOS: FX506HCB.316
Procesador: 11th Gen Intel(R) Core(TM) i5-11400H @ 2.70GHz (12 CPUs), ~2.7GHz
Memoria: 32768MB RAM
Archivo de paginación: 14699MB usados, 19855MB disponibles
Versión de DirectX: DirectX 12

Figura 1: Características computacionales del equipo utilizado para los experimentos.

Mientras el equipo de la supercomputadora LAVIS cuenta con las siguientes características computacionales:

- 6 nodos de computo con 186 procesadores en todos los nodos en conjunto.
- Procesador: Intel (R) Xeon (R) E3-1230 (3.50GHz)
- Memoria RAM (Nodo): 16 GB de RAM
- Sistema operativo: Rocky Linux
- Lenguaje de programación: C++

2. Desarrollo

Para el desarrollo de este trabajo se implementaron en C++ los algoritmos vistos en clase para resolver el problema de la subsecuencia de suma máxima, recordemos que tienen complejidades temporales distintas, las cuales son:

- Algoritmo 1: Complejidad $O(n^3)$
- Algoritmo 2: Complejidad $O(n^2)$
- Algoritmo 3: Complejidad $O(n \log n)$
- Algoritmo 4: Complejidad $O(n)$
- Algoritmo 5: Complejidad $O(n)$

Una vez implementados los algoritmos, se verificó su correcto funcionamiento utilizando casos de prueba con instancias pequeñas con pruebas apartir de $n = 10$ y aumentando progresivamente hasta tamaños mayores, manteniendo siempre la misma instancia de datos para todos los algoritmos con el fin de garantizar una comparación justa de los tiempos de ejecución.

Se obtuvieron los siguientes resultados para cada algoritmo:

2.1. Computadora Local (Propia)

Secuencial 1 Nucleo					
10	100	1000	10000	20000	
0	0.001	0.281	281.478	2333.71	
0	0	0.002	0.153	0.627	
0	0	0	0.001	0.002	
0	0	0	0	0	

Figura 2: Tiempo en segundos obtenido para cada algoritmo en computadora local.

Paralelo con 6 Nucleos					
10	100	1000	10000	20000	
0	0.001	0.313	288.895	2446.27	
0	0	0.002	0.138	0.592	
0	0	0	0.001	0.002	
0	0	0	0	0	

Figura 4: Tiempo en segundos obtenido para cada algoritmo en computadora local.

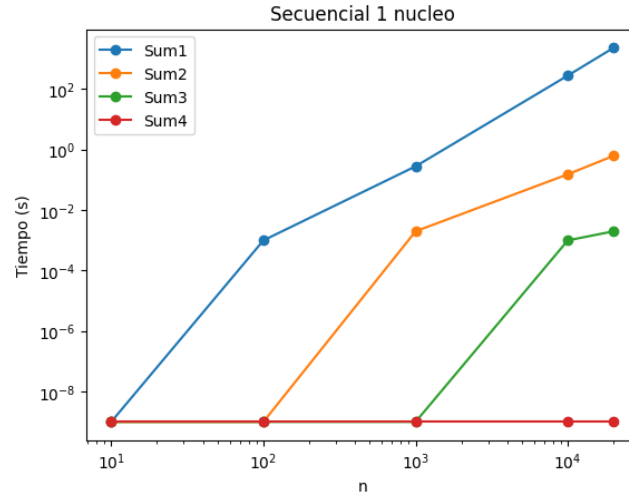


Figura 3: Grafica en escala logaritmica para cada algoritmo en computadora local.

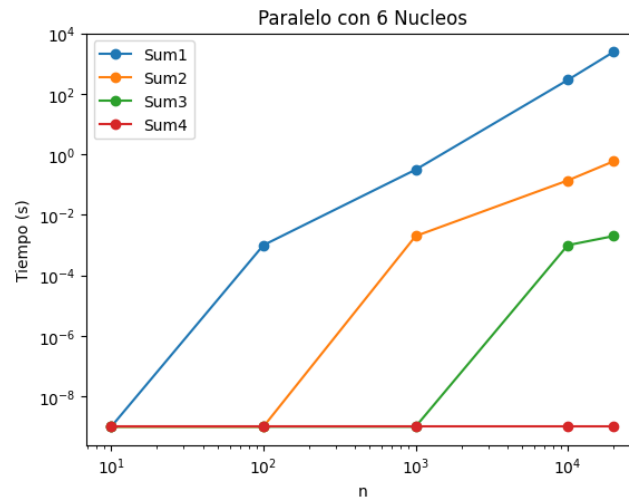


Figura 5: Grafica en escala logaritmica para cada algoritmo en computadora local.

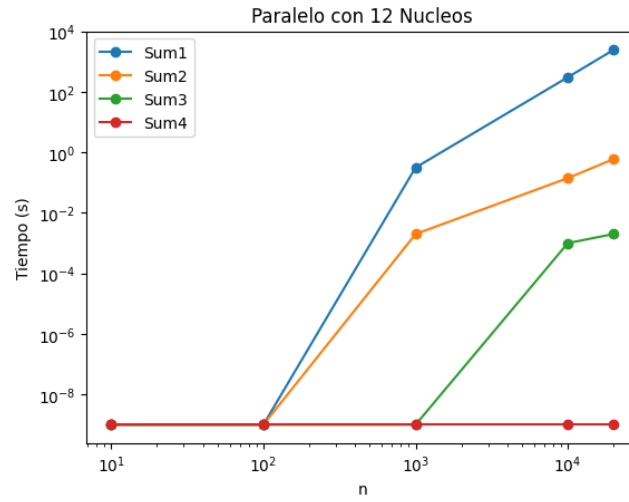


Figura 7: Grafica en escala logaritmica para cada algoritmo en computadora local.

Paralelo con 12 Nucleos					
	10	100	1000	10000	20000
Sum1	0	0	0.315	302.994	2429.49
Sum2	0	0	0.002	0.141	0.598
Sum3	0	0	0	0.001	0.002
Sum4	0	0	0	0	0

Figura 6: Tiempo en segundos obtenido para cada algoritmo en computadora local.

2.2. Supercomputadora LAVIS

Lineal 1 nucleo						
Algoritmo	10	100	1000	10000	20000	40000
Sum1	3.00E-06	0.000886	0.748477	749.396	2446.27	48573.6
Sum2	2.00E-06	4.70E-05	0.003959	0.390658	0.592	6.35356
Sum3	3.00E-06	1.80E-05	0.000154	0.001698	0.002	0.007051
Sum4	1.00E-06	2.00E-06	1.00E-05	8.60E-05	1.67E-04	3.70E-04

Figura 8: Tiempo en segundos obtenido para cada algoritmo en computadora local.

Paralelo 6 nucleos					
Algoritmo	20k	40k	80k	100k	
Sum1	2.40E+02	2000.16	15138.5	28786.5	
Sum2	1.66E-01	8.77E-01	2.71941	4.05867	
Sum3	4.60E-04	4.73E-03	0.005508	0.005376	
Sum4	2.60E-05	5.10E-05	9.00E-05	1.12E-04	

Figura 10: Tiempo en segundos obtenido para cada algoritmo en computadora local.

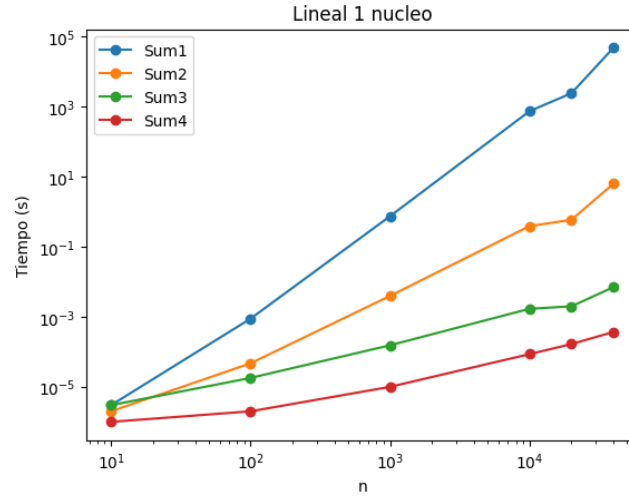


Figura 9: Grafica en escala logaritmica para cada algoritmo en computadora local.

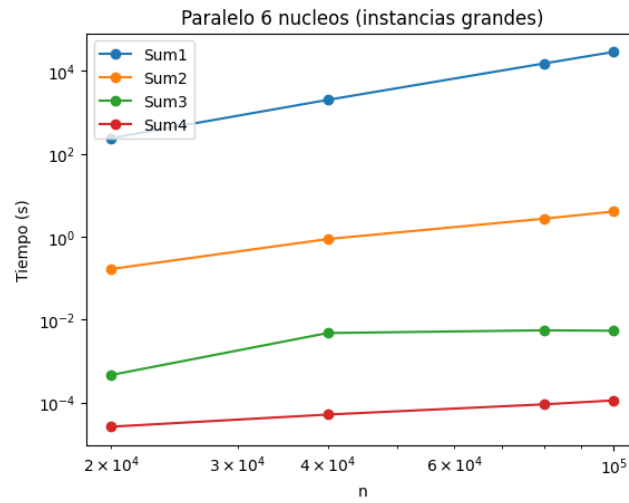


Figura 11: Grafica en escala logaritmica para cada algoritmo en computadora local.

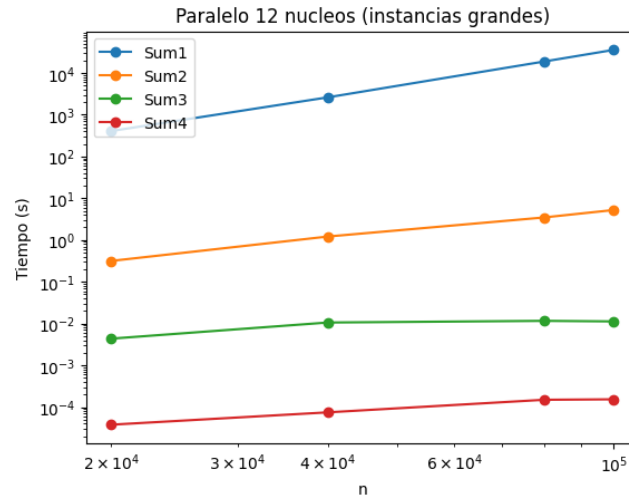


Figura 13: Grafica en escala logarítmica para cada algoritmo en computadora local.

Paralelo 12 nucleos				
Algoritmo	20k	40k	80k	100k
Sum1	4.06E+02	2595.01	18714.8	35520.2
Sum2	3.16E-01	1.21E+00	3.46818	5.22308
Sum3	4.36E-03	1.06E-02	0.011622	0.011275
Sum4	3.80E-05	7.50E-05	1.50E-04	1.54E-04

Figura 12: Tiempo en segundos obtenido para cada algoritmo en computadora local.

3. Conclusión

A partir de los experimentos realizados y de la gráfica comparativa en escala logarítmica, es posible observar claramente el comportamiento de los cuatro algoritmos analizados para el problema de la subsecuencia de suma máxima.

- **Sum1** ($O(n^3)$) presenta el crecimiento más rápido en el tiempo de ejecución. Conforme aumenta el tamaño de la instancia n , el tiempo requerido crece de manera muy acelerada, lo que lo vuelve impráctico para valores grandes de n .
- **Sum2** ($O(n^2)$) mejora considerablemente respecto al primer algoritmo. Aunque el crecimiento sigue siendo significativo, el tiempo de ejecución aumenta de manera menos pronunciada que en el caso cúbico.
- **Sum3** ($O(n \log n)$), basado en la estrategia de divide y vencerás, muestra una mejora importante en eficiencia. El crecimiento del tiempo de ejecución es mucho más moderado en comparación con los algoritmos anteriores.
- **Sum4** ($O(n)$), correspondiente al algoritmo lineal (Kadane), resulta ser el más eficiente de todos los algoritmos evaluados. Su tiempo de ejecución crece de forma casi proporcional al tamaño del arreglo y se mantiene muy pequeño incluso para instancias grandes.

Asimismo, los resultados obtenidos permiten observar que:

- La paralelización no modifica la complejidad asintótica de los algoritmos, pero sí reduce el tiempo de ejecución al disminuir los factores constantes asociados al cómputo.
- El algoritmo **Sum4** continúa siendo el más rápido incluso en ejecución secuencial, lo cual confirma el análisis teórico de su complejidad lineal.

En general, los resultados experimentales concuerdan con el análisis teórico de complejidad de cada algoritmo, evidenciando cómo el diseño algorítmico influye directamente en el desempeño al aumentar el tamaño de las instancias.